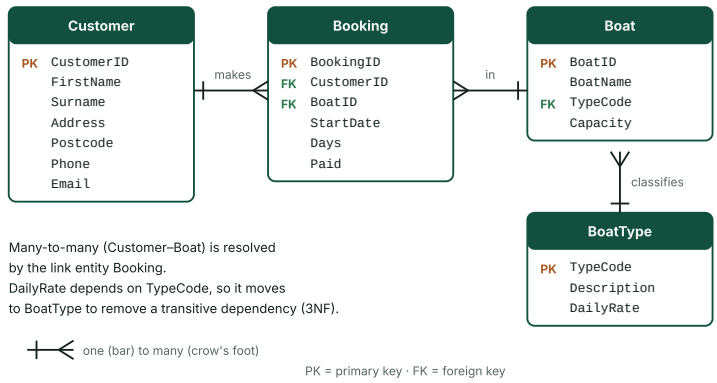
Design: ER diagrams, normalisation, data dictionary and interfaces

AD03 FACT FILE

Specification: students should be able to

- Design a relational database using an entity relationship diagram and normalised tables in third normal form
- Produce a data dictionary showing field names, data types, sizes, validation and keys
- Design user interfaces, reports, queries and algorithms that meet the requirements

Key words: entity, attribute, relationship, ER diagram, primary key, foreign key, normalisation, third normal form, data dictionary, validation rule, wireframe, query design



An ER diagram for Erne Boat Hire in third normal form: Customer and Boat each have a one-to-many relationship with Booking, and BoatType has a one-to-many relationship with Boat

ER diagrams and keys

An entity relationship diagram shows the entities in a system and the relationships between them. Relationships are one-to-one, one-to-many or many-to-many; relational databases cannot implement many-to-many directly, so a link entity is introduced, such as Booking linking Customer and Boat. Each table has a primary key that uniquely identifies each record. A foreign key is a field in one table that is the primary key of another, creating the relationship and allowing referential integrity to be enforced.

Normalisation

Normalisation organises data into tables to remove redundancy and avoid update, insertion and deletion anomalies. First normal form removes repeating groups and makes every field atomic. Second normal form removes partial dependencies, where a non-key attribute depends on only part of a composite key. Third normal form removes transitive dependencies, where a non-key attribute depends on another non-key attribute. A common summary is that every non-key attribute depends on the key, the whole key and nothing but the key.

Data dictionary and interface design

A data dictionary records, for every field, its name, data type, size or format, validation rule, whether it is a primary or foreign key, and a description or example. It ensures consistency and gives the developer exact instructions. Interface design uses annotated wireframes of forms, reports and menus showing layout, controls, navigation, validation and error messages. Designs should suit the users identified in analysis and reference the requirements they meet. Queries and algorithms are designed before they are built.

T Key terms from the mark schemes

Entity relationship diagram
A diagram showing the entities in a system and the one-to-one, one-to-many or many-to-many relationships between them.

Foreign key
A field in one table that is the primary key of another table, used to create a relationship between them.

Normalisation
The process of organising data into related tables to remove redundancy and prevent update, insertion and deletion anomalies.

Third normal form
A table is in 3NF when it is in 2NF and no non-key attribute depends on another non-key attribute.

Data dictionary
A document recording each field's name, data type, size, validation rule, key status and description.

Wireframe
A labelled sketch of the layout of a form, report or screen showing its controls and navigation before it is built.

Exam focus

Show the normalisation process, not just the final tables, and explain each dependency you removed. Make sure your ER diagram, tables and data dictionary agree with each other: inconsistent field names are a common weakness. Every field needs a sensible type and size, and validation must be specific, such as 'between 1 and 14' rather than 'range check'. Annotate wireframes rather than presenting blank sketches. Where you will use macros or VBA code, include pseudocode. Link designs to requirement numbers throughout.

? Check yourself

1. Define the term foreign key. [2]
2. Explain why a many-to-many relationship between Customer and Boat must be resolved, and how it is resolved. [3]
3. A Boat table contains BoatID, BoatName, TypeCode and DailyRate, where the daily rate depends on the type of boat. Explain which normal form is not satisfied and how to correct it. [4]

Answers: 1. A foreign key is a field (attribute) in one table that is the primary key of another table (1) / used to create the relationship between the two tables and allow referential integrity to be enforced, e.g. CustomerID in Booking (1). 2. A customer can hire many boats and a boat can be hired by many customers over time (1) / a relational database cannot implement M:N directly because a field cannot hold a list of keys – fields must be atomic (1) / it is resolved by a link entity, Booking, which holds CustomerID and BoatID as foreign keys, giving two one-to-many relationships: Customer 1–M Booking and Boat 1–M Booking (1). 3. Third normal form is not satisfied (1) / because there is a transitive dependency: DailyRate depends on TypeCode, which is a non-key attribute, rather than directly on the key BoatID (1) / correct it by moving DailyRate into a new table BoatType(TypeCode, Description, DailyRate) (1) / and leaving Boat(BoatID, BoatName, TypeCode*) with TypeCode as a foreign key linking to BoatType (1).